

LearnCpp.com is a free website devoted to teaching you how to program in modern C++. The lessons on this site will walk you through all the steps needed to write, compile, and debug your C++ programs. No prior programming experience is necessary, but programmers of all levels will benefit from our best practices, tips, and insights.

Becoming an expert won't happen overnight, but with a bit of patience, you'll get there. And LearnCpp.com will show you the way.

Introduction / Getting Started

Chapter 0

- 0.1 [Introduction to these tutorials](#)
- 0.2 [Introduction to programs and programming languages](#)
- 0.3 [Introduction to C/C++](#)
- 0.4 [Introduction to C++ development](#)
- 0.5 [Introduction to the compiler, linker, and libraries](#)
- 0.6 [Installing an Integrated Development Environment \(IDE\)](#)
- 0.7 [Compiling your first program](#)
- 0.8 [A few common C++ problems](#)
- 0.9 [Configuring your compiler: Build configurations](#)
- 0.10 [Configuring your compiler: Compiler extensions](#)
- 0.11 [Configuring your compiler: Warning and error levels](#)
- 0.12 [Configuring your compiler: Choosing a language standard](#)
- 0.13 [What language standard is my compiler using?](#)

C++ Basics

Chapter 1

- 1.1 [Statements and the structure of a program](#)
- 1.2 [Comments](#)
- 1.3 [Introduction to objects and variables](#)
- 1.4 [Variable assignment and initialization](#)

- 1.5 [Introduction to iostream: cout, cin, and endl](#)
- 1.6 [Uninitialized variables and undefined behavior](#)
- 1.7 [Keywords and naming identifiers](#)
- 1.8 [Whitespace and basic formatting](#)
- 1.9 [Introduction to literals and operators](#)
- 1.10 [Introduction to expressions](#)
- 1.11 [Developing your first program](#)
- 1.x [Chapter 1 summary and quiz](#)

C++ Basics: Functions and Files

Chapter 2

- 2.1 [Introduction to functions](#)
- 2.2 [Function return values \(value-returning functions\)](#)
- 2.3 [Void functions \(non-value returning functions\)](#)
- 2.4 [Introduction to function parameters and arguments](#)
- 2.5 [Introduction to local scope](#)
- 2.6 [Why functions are useful, and how to use them effectively](#)
- 2.7 [Forward declarations and definitions](#)
- 2.8 [Programs with multiple code files](#)
- 2.9 [Naming collisions and an introduction to namespaces](#)
- 2.10 [Introduction to the preprocessor](#)
- 2.11 [Header files](#)
- 2.12 [Header guards](#)
- 2.13 [How to design your first programs](#)
- 2.x [Chapter 2 summary and quiz](#)

Debugging C++ Programs

Chapter 3

- 3.1 [Syntax and semantic errors](#)
- 3.2 [The debugging process](#)
- 3.3 [A strategy for debugging](#)
- 3.4 [Basic debugging tactics](#)
- 3.5 [More debugging tactics](#)
- 3.6 [Using an integrated debugger: Stepping](#)
- 3.7 [Using an integrated debugger: Running and breakpoints](#)
- 3.8 [Using an integrated debugger: Watching variables](#)

3.9 [Using an integrated debugger: The call stack](#)

3.10 [Finding issues before they become problems](#)

3.x [Chapter 3 summary and quiz](#)

Fundamental Data Types

Chapter 4

4.1 [Introduction to fundamental data types](#)

4.2 [Void](#)

4.3 [Object sizes and the sizeof operator](#)

4.4 [Signed integers](#)

4.5 [Unsigned integers, and why to avoid them](#)

4.6 [Fixed-width integers and size_t](#)

4.7 [Introduction to scientific notation](#)

4.8 [Floating point numbers](#)

4.9 [Boolean values](#)

4.10 [Introduction to if statements](#)

4.11 [Chars](#)

4.12 [Introduction to type conversion and static_cast](#)

4.x [Chapter 4 summary and quiz](#)

Constants and Strings

Chapter 5

5.1 [Constant variables \(named constants\)](#)

5.2 [Literals](#)

5.3 [Numeral systems \(decimal, binary, hexadecimal, and octal\)](#)

5.4 [The as-if rule and compile-time optimization](#) Split

5.5 [Constant expressions](#) Updated

5.6 [Constexpr variables](#)

5.7 [Introduction to std::string](#)

5.8 [Introduction to std::string_view](#)

5.9 [std::string_view \(part 2\)](#)

5.x [Chapter 5 summary and quiz](#)

Operators

Chapter 6

6.1 [Operator precedence and associativity](#)

6.2 [Arithmetic operators](#)

- 6.3 [Remainder and Exponentiation](#)
- 6.4 [Increment/decrement operators, and side effects](#)
- 6.5 [The comma operator](#)
- 6.6 [The conditional operator](#) Moved
- 6.7 [Relational operators and floating point comparisons](#)
- 6.8 [Logical operators](#)
- 6.x [Chapter 6 summary and quiz](#)

Bit Manipulation (optional chapter)

Chapter O

- O.1 [Bit flags and bit manipulation via std::bitset](#)
- O.2 [Bitwise operators](#)
- O.3 [Bit manipulation with bitwise operators and bit masks](#)
- O.4 [Converting integers between binary and decimal representation](#)

Scope, Duration, and Linkage

Chapter 7

- 7.1 [Compound statements \(blocks\)](#)
- 7.2 [User-defined namespaces and the scope resolution operator](#)
- 7.3 [Local variables](#)
- 7.4 [Introduction to global variables](#)
- 7.5 [Variable shadowing \(name hiding\)](#)
- 7.6 [Internal linkage](#)
- 7.7 [External linkage and variable forward declarations](#)
- 7.8 [Why \(non-const\) global variables are evil](#)
- 7.9 [Inline functions and variables](#) Moved
- 7.10 [Sharing global constants across multiple files \(using inline variables\)](#)
- 7.11 [Static local variables](#)
- 7.12 [Scope, duration, and linkage summary](#)
- 7.13 [Using declarations and using directives](#)
- 7.14 [Unnamed and inline namespaces](#)
- 7.x [Chapter 7 summary and quiz](#)

Control Flow

Chapter 8

- 8.1 [Control flow introduction](#)
- 8.2 [If statements and blocks](#)

- 8.3 [Common if statement problems](#)
- 8.4 [Constexpr if statements](#)
- 8.5 [Switch statement basics](#)
- 8.6 [Switch fallthrough and scoping](#)
- 8.7 [Goto statements](#)
- 8.8 [Introduction to loops and while statements](#)
- 8.9 [Do while statements](#)
- 8.10 [For statements](#)
- 8.11 [Break and continue](#)
- 8.12 [Halts \(exiting your program early\)](#)
- 8.13 [Introduction to random number generation](#)
- 8.14 [Generating random numbers using Mersenne Twister](#)
- 8.15 [Global random numbers \(Random.h\)](#)
- 8.x [Chapter 8 summary and quiz](#)

Error Detection and Handling

Chapter 9

- 9.1 [Introduction to testing your code](#)
- 9.2 [Code coverage](#)
- 9.3 [Common semantic errors in C++](#)
- 9.4 [Detecting and handling errors](#)
- 9.5 [std::cin and handling invalid input](#)
- 9.6 [Assert and static_assert](#)
- 9.x [Chapter 9 summary and quiz](#)

Type Conversion, Type Aliases, and Type Deduction

Chapter 10

- 10.1 [Implicit type conversion](#)
- 10.2 [Floating-point and integral promotion](#)
- 10.3 [Numeric conversions](#)
- 10.4 [Narrowing conversions, list initialization, and constexpr initializers](#)
- 10.5 [Arithmetic conversions](#)
- 10.6 [Explicit type conversion \(casting\) and static_cast](#)
- 10.7 [Typedefs and type aliases](#)
- 10.8 [Type deduction for objects using the auto keyword](#)
- 10.9 [Type deduction for functions](#)

Function Overloading and Function Templates

Chapter 11

- 11.1 Introduction to function overloading
- 11.2 Function overload differentiation
- 11.3 Function overload resolution and ambiguous matches
- 11.4 Deleting functions
- 11.5 Default arguments
- 11.6 Function templates
- 11.7 Function template instantiation
- 11.8 Function templates with multiple template types
- 11.9 Non-type template parameters
- 11.10 Using function templates in multiple files
- 11.x Chapter 11 summary and quiz

Constexpr functions

Chapter F

- F.1 Cxxpr functions **Moved**
- F.2 Cxxpr functions (part 2) **Split**
- F.3 Cxxpr functions (part 3) and consteval **Split**
- F.4 Cxxpr functions (part 4) **Split**
- F.X Chapter F summary and quiz **Split**

Compound Types: References and Pointers

Chapter 12

- 12.1 Introduction to compound data types
- 12.2 Value categories (lvalues and rvalues)
- 12.3 Lvalue references
- 12.4 Lvalue references to const
- 12.5 Pass by lvalue reference
- 12.6 Pass by const lvalue reference
- 12.7 Introduction to pointers
- 12.8 Null pointers
- 12.9 Pointers and const
- 12.10 Pass by address
- 12.11 Pass by address (part 2)

[12.12 Return by reference and return by address](#)

[12.13 In and out parameters](#)

[12.14 Type deduction with pointers, references, and const](#)

[12.15 std::optional](#)

[12.x Chapter 12 summary and quiz](#)

Compound Types: Enums and Structs

Chapter 13

[13.1 Introduction to program-defined \(user-defined\) types](#)

[13.2 Unscoped enumerations](#)

[13.3 Unscoped enumerator integral conversions](#)

[13.4 Converting an enumeration to and from a string](#)

[13.5 Introduction to overloading the I/O operators](#)

[13.6 Scoped enumerations \(enum classes\)](#)

[13.7 Introduction to structs, members, and member selection](#)

[13.8 Struct aggregate initialization](#)

[13.9 Default member initialization](#)

[13.10 Passing and returning structs](#)

[13.11 Struct miscellany](#)

[13.12 Member selection with pointers and references](#)

[13.13 Class templates](#)

[13.14 Class template argument deduction \(CTAD\) and deduction guides](#)

[13.15 Alias templates](#)

[13.x Chapter 13 summary and quiz](#)

[13.y Using a language reference](#)

Introduction to Classes

Chapter 14

[14.1 Introduction to object-oriented programming](#)

[14.2 Introduction to classes](#)

[14.3 Member functions](#)

[14.4 Const class objects and const member functions](#)

[14.5 Public and private members and access specifiers](#)

[14.6 Access functions](#)

[14.7 Member functions returning references to data members](#)

[14.8 The benefits of data hiding \(encapsulation\)](#)

- 14.9 Introduction to constructors
- 14.10 Constructor member initializer lists
- 14.11 Default constructors and default arguments
- 14.12 Delegating constructors
- 14.13 Temporary class objects
- 14.14 Introduction to the copy constructor
- 14.15 Class initialization and copy elision
- 14.16 Converting constructors and the explicit keyword
- 14.17 Constexpr aggregates and classes
- 14.x Chapter 14 summary and quiz

More on Classes

Chapter 15

- 15.1 The hidden "this" pointer and member function chaining
- 15.2 Classes and header files
- 15.3 Nested types (member types)
- 15.4 Introduction to destructors
- 15.5 Class templates with member functions
- 15.6 Static member variables
- 15.7 Static member functions
- 15.8 Friend non-member functions
- 15.9 Friend classes and friend member functions
- 15.10 Ref qualifiers
- 15.x Chapter 15 summary and quiz

Dynamic arrays: `std::vector`

Chapter 16

- 16.1 Introduction to containers and arrays
- 16.2 Introduction to `std::vector` and list constructors
- 16.3 `std::vector` and the unsigned length and subscript problem
- 16.4 Passing `std::vector`
- 16.5 Returning `std::vector`, and an introduction to move semantics
- 16.6 Arrays and loops
- 16.7 Arrays, loops, and sign challenge solutions
- 16.8 Range-based for loops (for-each)
- 16.9 Array indexing and length using enumerators

[16.10 std::vector resizing and capacity](#)

[16.11 std::vector and stack behavior](#)

[16.12 std::vector<bool>](#)

[16.x Chapter 16 summary and quiz](#)

Fixed-size arrays: std::array and C-style arrays

Chapter 17

[17.1 Introduction to std::array](#)

[17.2 std::array length and indexing](#)

[17.3 Passing and returning std::array](#)

[17.4 std::array of class types, and brace elision](#)

[17.5 Arrays of references via std::reference_wrapper](#)

[17.6 std::array and enumerations](#)

[17.7 Introduction to C-style arrays](#)

[17.8 C-style array decay](#)

[17.9 Pointer arithmetic and subscripting](#)

[17.10 C-style strings](#)

[17.11 C-style string symbolic constants](#)

[17.12 Multidimensional C-style Arrays](#)

[17.13 Multidimensional std::array](#)

[17.x Chapter 17 summary and quiz](#)

Iterators and Algorithms (under construction)

Chapter 18

[18.1 Sorting an array using selection sort](#)

[18.2 Introduction to iterators](#)

[18.3 Introduction to standard library algorithms](#)

[18.4 Timing your code](#)

Dynamic Allocation (under construction)

Chapter 19

[19.1 Dynamic memory allocation with new and delete](#)

[19.2 Dynamically allocating arrays](#)

[19.3 Destructors](#)

[19.4 Pointers to pointers and dynamic multidimensional arrays](#)

[19.5 Void pointers](#)

Functions

Chapter 20

- 20.1 [Function Pointers](#)
- 20.2 [The stack and the heap](#)
- 20.3 [Recursion](#)
- 20.4 [Command line arguments](#)
- 20.5 [Ellipsis \(and why to avoid them\)](#)
- 20.6 [Introduction to lambdas \(anonymous functions\)](#)
- 20.7 [Lambda captures](#)
- 20.x [Chapter 20 summary and quiz](#)

Operator Overloading

Chapter 21

- 21.1 [Introduction to operator overloading](#)
- 21.2 [Overloading the arithmetic operators using friend functions](#)
- 21.3 [Overloading operators using normal functions](#)
- 21.4 [Overloading the I/O operators](#)
- 21.5 [Overloading operators using member functions](#)
- 21.6 [Overloading unary operators +, -, and !](#)
- 21.7 [Overloading the comparison operators](#)
- 21.8 [Overloading the increment and decrement operators](#)
- 21.9 [Overloading the subscript operator](#)
- 21.10 [Overloading the parenthesis operator](#)
- 21.11 [Overloading typecasts](#)
- 21.12 [Overloading the assignment operator](#)
- 21.13 [Shallow vs. deep copying](#)
- 21.14 [Overloading operators and function templates](#)
- 21.x [Chapter 21 summary and quiz](#)
- 21.y [Chapter 21 project](#)

Move Semantics and Smart Pointers

Chapter 22

- 22.1 [Introduction to smart pointers and move semantics](#)
- 22.2 [R-value references](#)
- 22.3 [Move constructors and move assignment](#)
- 22.4 [std::move](#)
- 22.5 [std::unique_ptr](#)

[22.6 `std::shared\_ptr`](#)

[22.7 Circular dependency issues with `std::shared\_ptr`, and `std::weak\_ptr`](#)

[22.x Chapter 22 summary and quiz](#)

Object Relationships

Chapter 23

[23.1 Object relationships](#)

[23.2 Composition](#)

[23.3 Aggregation](#)

[23.4 Association](#)

[23.5 Dependencies](#)

[23.6 Container classes](#)

[23.7 `std::initializer\_list`](#)

[23.x Chapter 23 summary and quiz](#)

Inheritance

Chapter 24

[24.1 Introduction to inheritance](#)

[24.2 Basic inheritance in C++](#)

[24.3 Order of construction of derived classes](#)

[24.4 Constructors and initialization of derived classes](#)

[24.5 Inheritance and access specifiers](#)

[24.6 Adding new functionality to a derived class](#)

[24.7 Calling inherited functions and overriding behavior](#)

[24.8 Hiding inherited functionality](#)

[24.9 Multiple inheritance](#)

[24.x Chapter 24 summary and quiz](#)

Virtual Functions

Chapter 25

[25.1 Pointers and references to the base class of derived objects](#)

[25.2 Virtual functions and polymorphism](#)

[25.3 The override and final specifiers, and covariant return types](#)

[25.4 Virtual destructors, virtual assignment, and overriding virtualization](#)

[25.5 Early binding and late binding](#)

[25.6 The virtual table](#)

[25.7 Pure virtual functions, abstract base classes, and interface classes](#)

25.8 [Virtual base classes](#)

25.9 [Object slicing](#)

25.10 [Dynamic casting](#)

25.11 [Printing inherited classes using operator<<](#)

25.x [Chapter 25 summary and quiz](#)

Templates and Classes

Chapter 26

26.1 [Template classes](#)

26.2 [Template non-type parameters](#)

26.3 [Function template specialization](#)

26.4 [Class template specialization](#)

26.5 [Partial template specialization](#)

26.6 [Partial template specialization for pointers](#)

26.x [Chapter 26 summary and quiz](#)

Exceptions

Chapter 27

27.1 [The need for exceptions](#)

27.2 [Basic exception handling](#)

27.3 [Exceptions, functions, and stack unwinding](#)

27.4 [Uncaught exceptions and catch-all handlers](#)

27.5 [Exceptions, classes, and inheritance](#)

27.6 [Rethrowing exceptions](#)

27.7 [Function try blocks](#)

27.8 [Exception dangers and downsides](#)

27.9 [Exception specifications and noexcept](#)

27.10 [std::move_if_noexcept](#)

27.x [Chapter 27 summary and quiz](#)

Input and Output (I/O)

Chapter 28

28.1 [Input and output \(I/O\) streams](#)

28.2 [Input with istream](#)

28.3 [Output with ostream and ios](#)

28.4 [Stream classes for strings](#)

[28.5 Stream states and input validation](#)

[28.6 Basic file I/O](#)

[28.7 Random file I/O](#)

Miscellaneous Subjects

Appendix A

[A.1 Static and dynamic libraries](#)

[A.2 Using libraries with Visual Studio](#)

[A.3 Using libraries with Code::Blocks](#)

[A.4 C++ FAQ](#)

C++ Updates

Appendix B

[B.1 Introduction to C++11](#)

[B.2 Introduction to C++14](#)

[B.3 Introduction to C++17](#)

[B.4 Introduction to C++20](#)

[B.5 Introduction to C++23](#)

The End

Appendix C

[C.1 The end?](#)

Deprecated Articles (will be removed soon)

Appendix D

[21.1 The Standard Library](#)

[21.2 STL containers overview](#)

[21.3 STL iterators overview](#)

[21.4 STL algorithms overview](#)

[22.1 std::string and std::wstring](#)

[22.2 std::string construction and destruction](#)

[22.3 std::string length and capacity](#)

[22.4 std::string character access and conversion to C-style arrays](#)

[22.5 std::string assignment and swapping](#)

[22.6 std::string appending](#)

[22.7 std::string inserting](#)